

01. 根据视频 (041ResNet代码实现1、041ResNet代码实现2)，请把网络处理“img”图像时每个卷积层的输出形状推算出来。

答：

```
PS C:\TZ\Code\week5\resnet> & "C:/Program Files/Python311/python.exe" c:/TZ/Code/week5/resnet/resnet.py
torch.Size([4, 3, 6, 6])
torch.Size([4, 6, 3, 3])
tensor([[[-0.1367, -0.2994, -0.2405, -0.0775,  0.0759,  0.1254,  0.0624, -0.6432,
          0.0442, -0.0197]], grad_fn=<AddmmBackward0>)]
Sequential output shape:      torch.Size([1, 64, 56, 56])
Sequential output shape:      torch.Size([1, 64, 56, 56])
Sequential output shape:      torch.Size([1, 128, 28, 28])
Sequential output shape:      torch.Size([1, 256, 14, 14])
Sequential output shape:      torch.Size([1, 512, 7, 7])
AdaptiveAvgPool2d output shape: torch.Size([1, 512, 1, 1])
Flatten output shape:          torch.Size([1, 512])
Linear output shape:           torch.Size([1, 10])
```

02. 如图：

视频 (041ResNet代码实现2) 中nn.Linear(512, 10)网络最后的全连接层为什么参数形状为512\*10？决定这个形状的因素是什么？

答：

512 是由最后一个卷积层（b5）的输出通道数决定的，10 是由分类任务决定的。

```
b5 = nn.Sequential(*resnet_block(256, 512, 2))
```

是网络中最后一个残差块组。resnet\_block 函数创建的 Residual 块会将输入通道数从 256 转换成输出通道数 512。所以，从 b5 出来的特征图（Tensor）的形状是 (批大小, 512, 7, 7)。

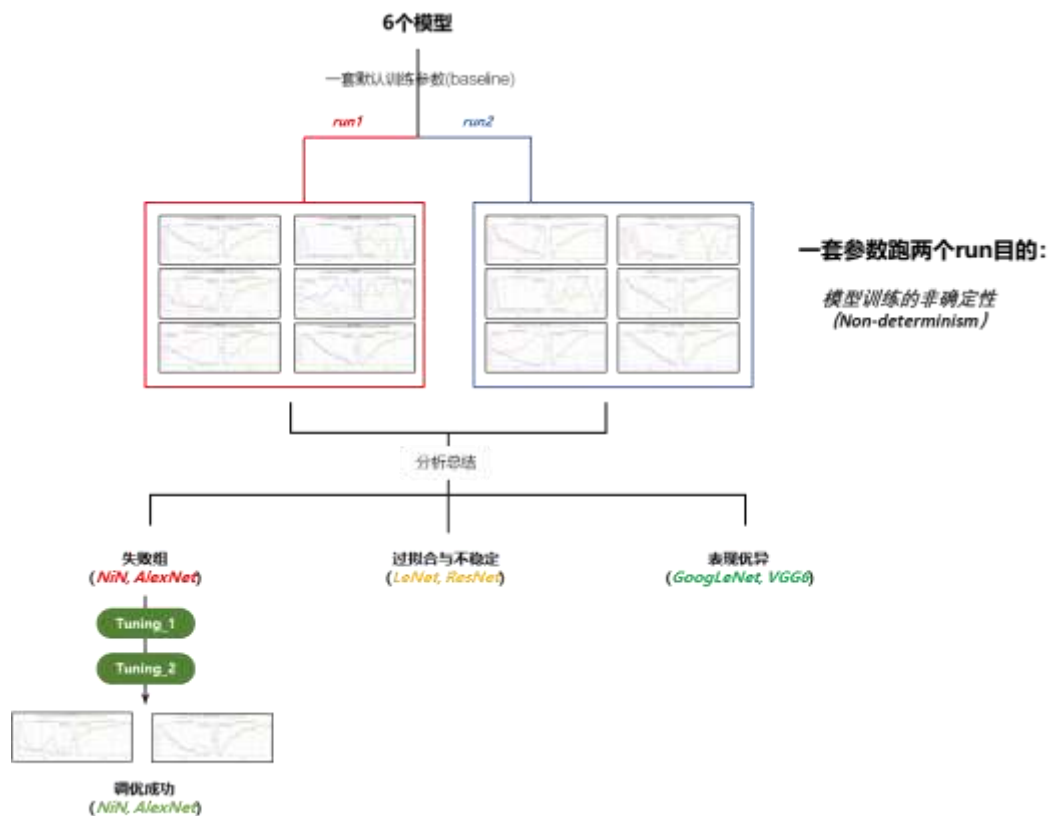
**nn.AdaptiveAvgPool2d((1, 1)):** 这是一个自适应平均池化层。它的作用是，无论输入的高和宽是多少，都将它们池化成 (1, 1) 的大小。所以，经过这一层后，特征图的形状就变成了 (批大小, 512, 1, 1)。

**nn.Flatten():** 这一层会将多维的张量“压平”成一个二维张量。它会保留第 0 维（批大小），然后将后面的所有维度相乘。所以，(批大小, 512, 1, 1) 的张量被压平成了 (批大小, 512 \* 1 \* 1)，也就是 (批大小, 512)。

**nn.Linear(512, 10):** 现在，输入到 nn.Linear 层的数据是一个形状为 (批大小, 512) 的张量。对于批次中的每一个样本，都有 512 个特征。因此，nn.Linear 层的输入特征数必须是 512。

### 03. 实战作业：

### 3.1 一张图看清我在第三题做了什么



我做了什么：基于 100x100 人脸数据集（Male/Female 二分类），对比不同模型架构的性能差异，我主要做了两件事情：

1. 针对 6 个模型，设置了一个默认参数默认基准参数（baseline），用这一套参数训练 6 个模型，用于比较不同模型在这个人脸分类问题上的性能差异，我 run 了两次所以有 run\_1 和 run\_2，这是为了测试模型训练的非确定性。
2. 针对 baseline 里面的“失败模型”，也就是 NiN 和 AlexNet，我对其做了两次调优（tuning\_1 和 tuning\_2）让 val\_acc 从约 50%的准确率跃升到 85%左右。

默认基准参数（baseline）如下：

| 参数名                | 值     | 说明                |
|--------------------|-------|-------------------|
| Optimizer          | Adam  | 优化器               |
| Learning Rate (LR) | 0.001 | 学习率               |
| Batch Size         | 16    | 批次大小 (已调整以避免显存溢出) |

|             |           |                                    |
|-------------|-----------|------------------------------------|
| Epochs      | 10        | 训练轮次                               |
| Split Ratio | 0.8       | 训练集占比 (80% 训练, 20% 验证)             |
| Image Size  | 100 x 100 | 输入图片尺寸 (RGB 3 通道)                  |
| Dropout     | 0.5       | Dropout 概率 (ResNet, VGG, AlexNet ) |

### 3.2 综合表现概览 (Executive Summary)

本次实验清晰地将 6 个模型划分为了三个明显的梯队。实验结果表明，在小尺寸（100x100）且数据量有限的任务中，“深层网络+小卷积核”（GoogLeNet, VGG6）的表现显著优于浅层网络”（LeNet）和“原始大图网络”（AlexNet, NiN）。

| Model_Name | Total_Training_Time | Final_Train_Acc    | Final_Val_Acc      |
|------------|---------------------|--------------------|--------------------|
| LeNet      | 45.1968035697937    | 0.9878760664571172 | 0.7881508078994613 |
| AlexNet    | 265.5924346446991   | 0.4984283789851819 | 0.4883303411131059 |
| VGG6       | 101.49052357673645  | 0.8837000449034575 | 0.8258527827648114 |
| NiN        | 164.6676197052002   | 0.5020206555904805 | 0.4883303411131059 |
| GoogLeNet  | 53.3822238445282    | 0.8778625954198473 | 0.8707360861759426 |
| ResNet     | 188.85864353179932  | 0.877413560844185  | 0.7594254937163375 |

Figure 1 六个模型在人脸识别上的表现

Baseline以run2为例

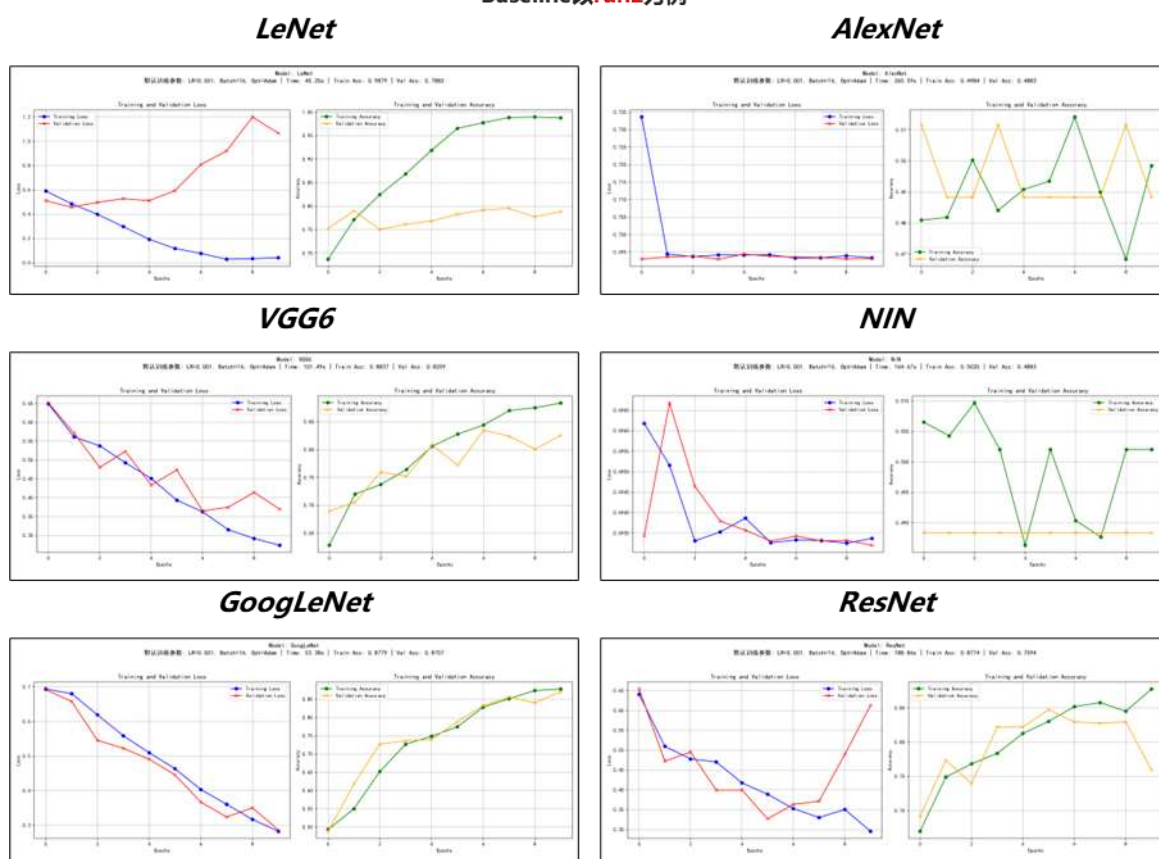


Figure 2 六个模型在人脸识别上的训练表现

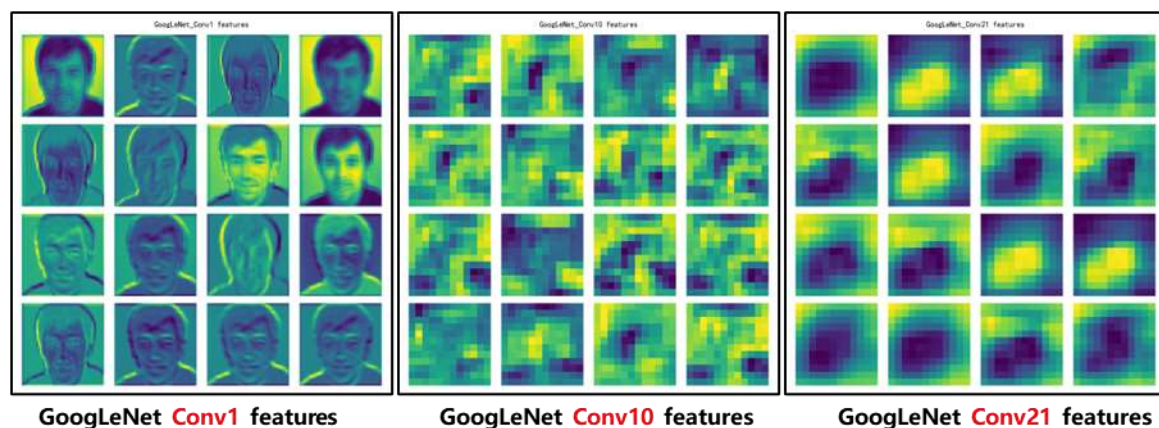


Figure 3 GoogLeNet 为例取深-中-浅三层的卷积层特征图

通过 GoogLeNet 为例取深-中-浅三层的卷积层特征图发现，越靠近输入层的（越浅层）学到的特征越具体，越像“男人”，中间卷积层学到的已经非常抽象了，深层网络学到的就更加抽象了，完全看不出是什么。

### 3.2.1 最佳模型 GoogLeNet（验证集准确率 ~87%）

特点：训练最稳，泛化能力最强，速度较快。

### 3.2.2 稳健模型：VGG6（验证集准确率 ~82%）

特点：结构简单有效，但全连接层参数过多导致训练稍慢。

### 3.2.3 过拟合/不稳定模型：LeNet, ResNet

**LeNet:** 典型的过拟合（训练 99% vs 验证 79%）。

**ResNet:** 严重的训练震荡，存在过拟合拐点。

### 3.2.4 失败模型：NiN, AlexNet

特点：无法收敛，准确率停滞在 50%（随机猜测）。

## 3.3 各模型详细分析

### 3.3.1 第一梯队：表现优异 (GoogLeNet, VGG6)

#### GoogLeNet (Inception v1)

- **表现分析：**Run 1 和 Run 2 均表现出最高的验证集准确率。Loss 曲线下降平滑，且验证集准确率（黄线）紧跟训练集准确率（绿线），说明 Inception 模块的多尺度特征提取在 100x100 这种中等分辨率图片上极其有效。
- **成功原因：**Inception 块并行的卷积操作既保留了局部细节，又提取了全局特征，且参数量控制得当。

#### VGG6 (Custom)

- **表现分析：**准确率稳定上升，没有剧烈的 Loss 震荡。
- **成功原因：**定制的 VGG6 采用了 3x3 小卷积核堆叠，非常适合 100x100 的输入。相比 AlexNet，它保留了更多的空间特征。
- **不足：**全连接层（4096 节点）过于庞大，导致参数冗余，训练耗时（~101s）是 GoogLeNet 的两倍。

### 3.3.2 第二梯队：过拟合与不稳定 (LeNet, ResNet)

#### LeNet

- **表现分析：**Run 2 中，训练准确率在 Epoch 5 就冲到了 96%，最后达到 98.79%，而验证准确率卡在 78%。这是典型的**过拟合**。
- **原因：**模型对于二分类任务来说容量足够，但缺乏约束，导致它“死记硬背”了训练集。

ResNet

- **表现分析：**Run 2 的 Loss 曲线在 Epoch 5 达到最低点，随后验证集 Loss（红线）剧烈反弹。
- **原因：学习率 (LR=0.001) 过高。**ResNet 引入了 BN 层，虽然有助于收敛，但在如果 LR 不下降，模型会跳出局部最优解（Loss 震荡）。同时，ResNet 模型较深，在小数据上容易过拟合。

3.3.3 第三梯队：完全失败 (NiN, AlexNet)

NiN (Network in Network)和 AlexNet 都表现出难以收敛，~50%的准确率约等于瞎猜性别分类，通过对网络结构和超参数进行综合分析后得出结论如下：

- 对于 NIN，通过检查发现最后一个 Block 的 ReLU 导致“信息截断”，严重干扰了 CrossEntropyLoss 的计算，导致梯度计算错误。
- 对于 Alex，通过检查发现全连接层过于臃肿（Parameter Explosion）、缺乏 Batch Normalization (BN)是主要原因

3.4 针对 2 个典型的“失败”模型的调优进行学习(NiN, AlexNet)

| 模型      | Run1/2<br>(baseline) | Tuning_1 改动                                             | Tuning_1    | Tuning_2 改动                                          | Tuning_2    |
|---------|----------------------|---------------------------------------------------------|-------------|------------------------------------------------------|-------------|
| NIN     | 50%（不收敛）             | 1.构建 ninblock 中加入 BN<br>2.构建 NINNet 中的最后一个 block 选择手动编写 | 84% val_acc | 调低学习率 lr 从 0.001 降低到 0.0001，增加权重衰减 weight_decay=1e-4 | 89% val_acc |
| AlexNet | 50%(不收敛)             | 1.全连接层参数量修改<br>2.加入 BN                                  | 82% val_acc | Dropout 从 0.5 调大到 0.7                                | 87% val_acc |

Figure 4 针对 Alex 和 NIN 的两次调优参数一览表

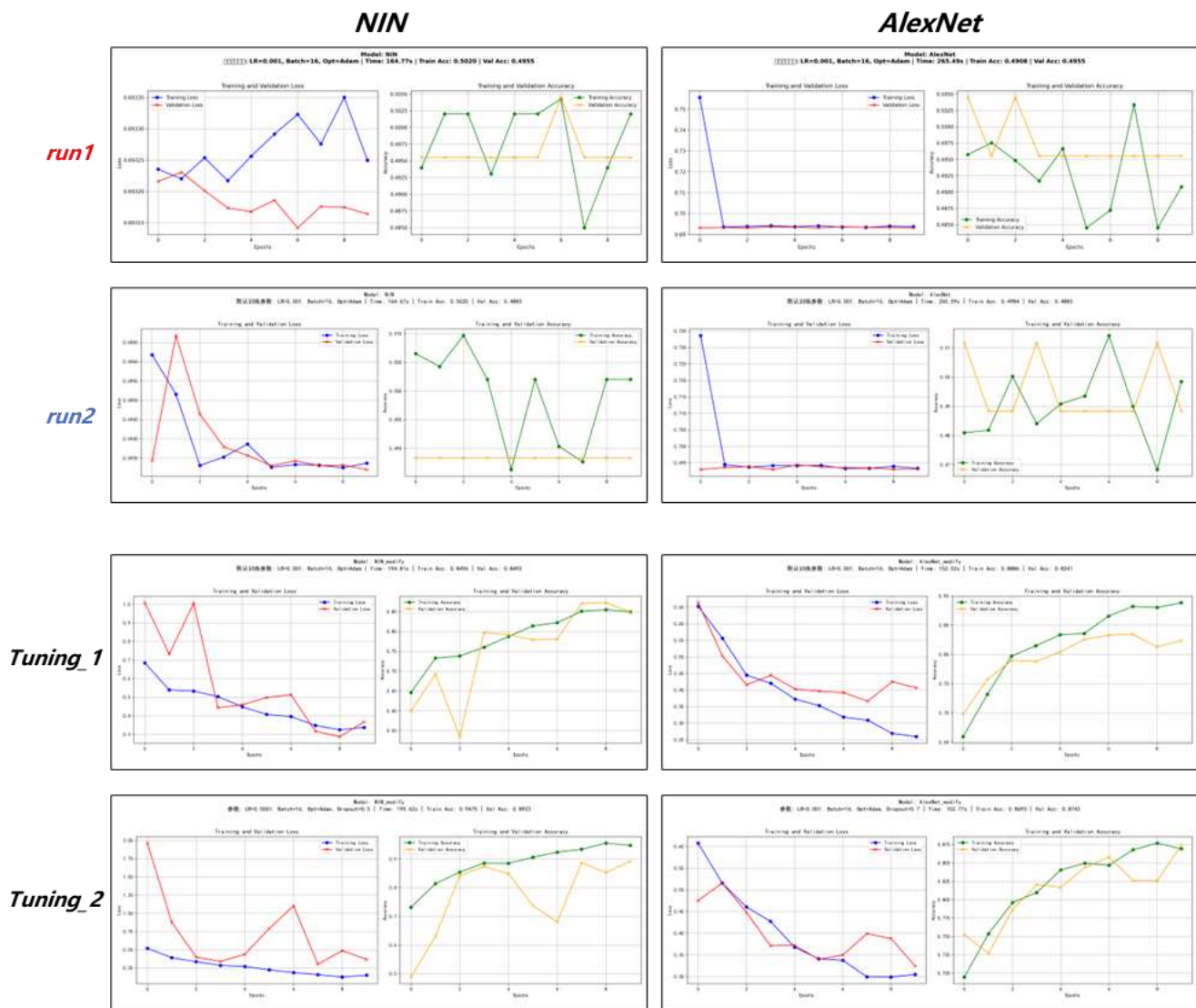


Figure 5 针对 Alex 和 NIN 的两次调优训练效果一览表

对 NIN 网络的调优 (Tuning\_1)

1. 构建 ninblock 中加入 BN。网络非常深，容易出现梯度消失和“死寂现象”，故加入 BN。



```

1 import torch
2 from torch import nn
3
4 def ninblock(in_channels, out_channels, kernel_size, stride, padding):
5     return nn.Sequential(
6         nn.Conv2d(in_channels, out_channels, kernel_size, stride, padding),
7         nn.ReLU(),
8         nn.Conv2d(out_channels, out_channels, kernel_size=1,
9         nn.ReLU(),
10        nn.Conv2d(out_channels, out_channels, kernel_size=1,
11        nn.ReLU())
12    )
13
14 class NIN(nn.Module):
15     def __init__(self, num_classes=2):
16         super(NIN, self).__init__()
17         self.net = nn.Sequential(
18             ninblock(3, 96, kernel_size=5, stride=1, padding=2),
19             nn.MaxPool2d(3, stride=2),
20             ninblock(96, 256, kernel_size=5, stride=1, padding=2),
21             nn.MaxPool2d(3, stride=2),
22             ninblock(256, 384, 3, 1, 1),
23             nn.MaxPool2d(3, stride=2),
24             nn.Dropout(0.5),
25             ninblock(384, num_classes, 3, 1, 1),
26             nn.AdaptiveAvgPool2d((1, 1)),
27             nn.Flatten()
28         )
29

```

Figure 6 NIN 网络深度很深 (3\*4=12 层卷积)

2.构建 NINNet 中的最后一个 block 选择手动编写。构建 Net 的时候最后一个 Block 的 Relu 会导致“信息截断”，如果模型认为某张图“非常不像男性”，它想输出一个负分（比如 -10），结果被 ReLU 强制变成了 0，这种现象严重干扰了 CrossEntropyLoss 的计算，导致梯度计算错误

```

4 class NIN(nn.Module):
5     def __init__(self, num_classes=2):
6         super(NIN, self).__init__()
7         self.net = nn.Sequential(
8             ninblock(3, 96, kernel_size=5, stride=1, padding=2),
9             nn.MaxPool2d(3, stride=2),
10            ninblock(96, 256, kernel_size=5, stride=1, padding=2),
11            nn.MaxPool2d(3, stride=2),
12            ninblock(256, 384, 3, 1, 1),
13            nn.MaxPool2d(3, stride=2),
14            nn.Dropout(0.5),
15            ninblock(384, num_classes, 3, 1, 1),
16            nn.AdaptiveAvgPool2d((1, 1)),
17            nn.Flatten()
18        )
19

```

Figure 7 NINnet 构建过程中最后一个 Block 也使用到了 relu



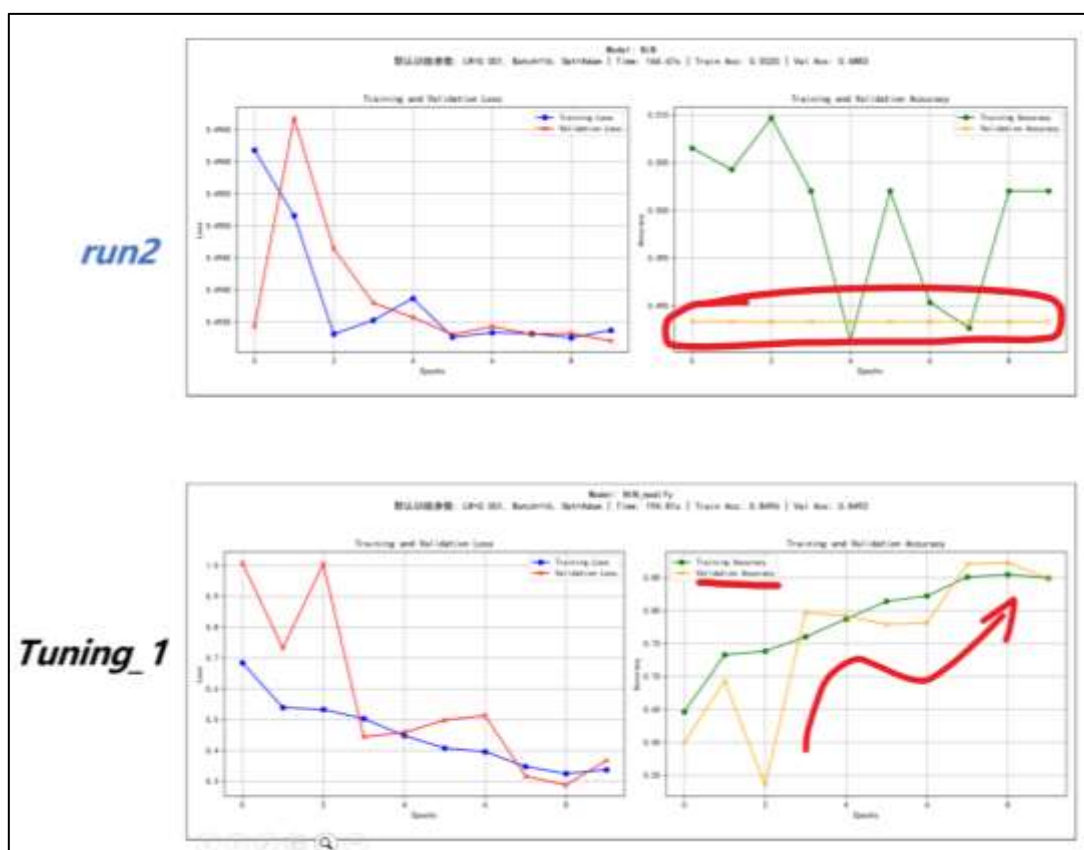


Figure 8 tuning\_1 后的 NIN 表现明显得到极大的改善

对 NIN 网络的调优 (Tuning\_2)

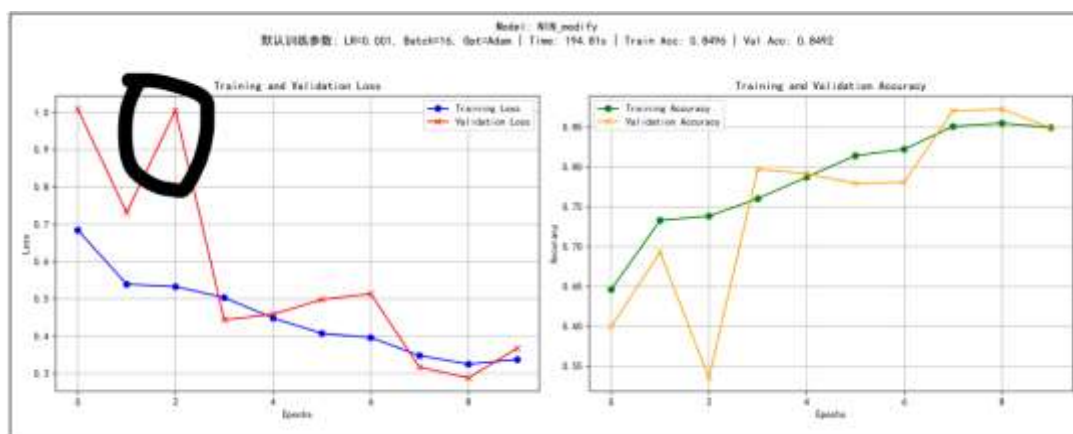


Figure 9 NIN 因为 Lr 太大损失函数初期下降异常的跳点现象

对于 tuning\_1 里面的 Val Loss, 在 Epoch 2 突然飙升到 1.0 以上, 然后迅速跌回 0.4, 猜测是因为学习率 (LR=0.001) 在初期对 NiN 这种深层结构来说稍大, 导致参数更新时迈了一大步, “跳”到了损失函数的一个高点。所以准备调低学习率 lr 从 0.001 降低到 0.0001, 并且增加权重衰减 weight\_decay=1e-4

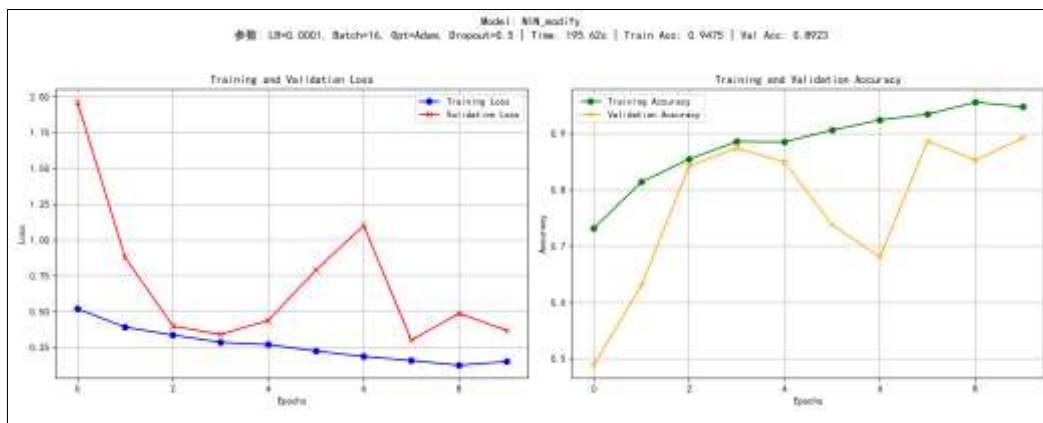


Figure 10 tuning\_2 的 NIN 训练表现（初始损失下降大大改善）

### 对 AlexNet 网络的调优（Tuning\_1）

通过检查 run\_2 跑的 baseline 代码发现，AlexNet 对于接入全连接层前的输入特征图大小是  $256 \times 11 \times 11$ ，展平后有 30,976 个神经元，连接到第一个全连接层（4096 节点），需要训练的权重参数数量为： $30,976 \times 4096 \approx 1.26$  亿 (126 Million)，所以一方面是全连接层 4096 参数量太大，决定把 4096 改成 512，并且加入了 BN 防止梯度小时。

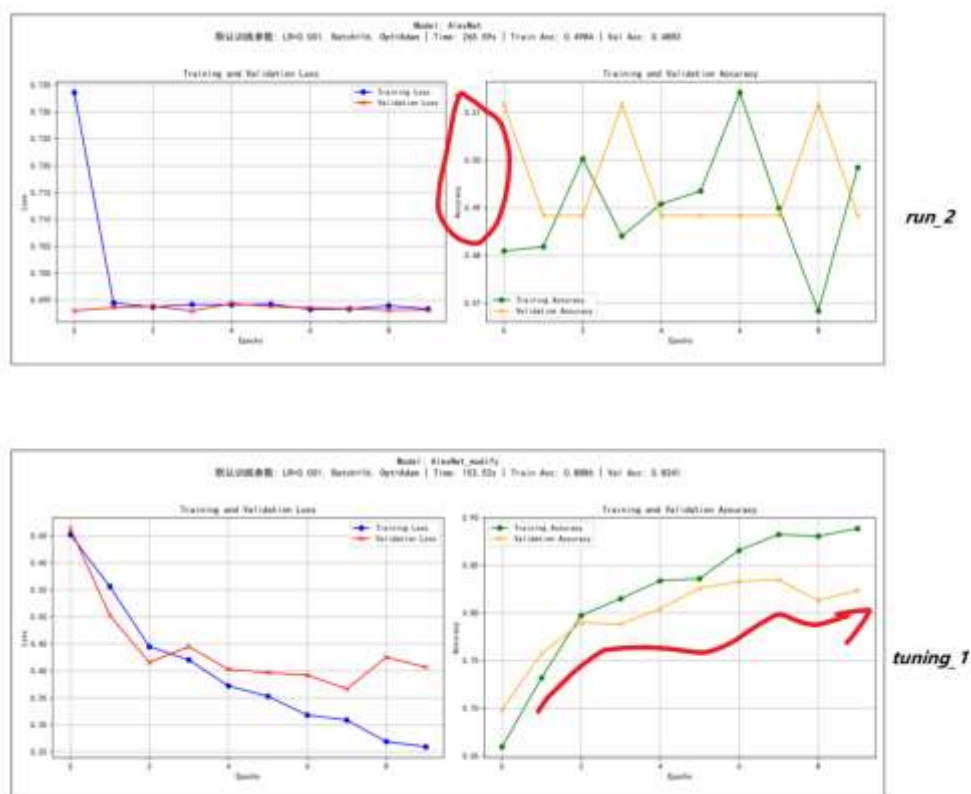


Figure 11 AlexNet 经过 tuning\_1 得到极大改善

## 对 AlexNet 网络的调优 (Tuning\_2)



Figure 12 AlexNet\_tuning\_1 的可能存在的过拟合

对于 AlexNet\_tuning\_1，它的 Loss 曲线非常丝滑，收敛速度也很完美，但在最后几个 Epoch (Epoch 7-9)，验证集 Loss 开始反弹 (从 0.36 涨到 0.40)，验证集准确率也停滞不前，存在轻微过拟合，所以准备增加“遗忘”的概率，将 Dropout 从 0.5 调大到 0.7。

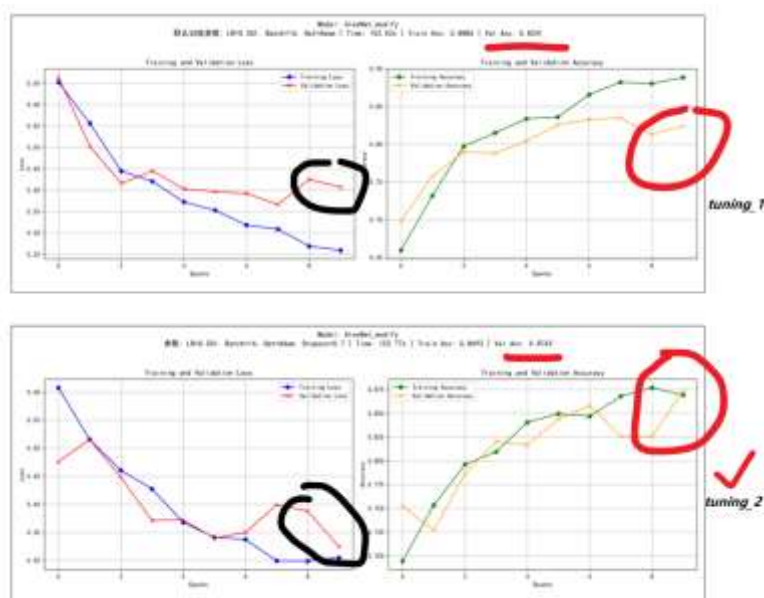


Figure 13 AlexNet\_tuning\_2 增大 dropout 遗忘策略起效果了